

Coursera

UNIVERSITY OF TORONTO

AI Agents and Agentic AI with Python & Generative AI

Module 1

Agentic AI Concepts

Module 2

AI Agents, Tools, Actions, & Language

Module 3

GAME, A Conceptual Framework for AI Agents

Overview of the GAME Framework
Video • 5 min

Designing AI Agents with GAME
Ungraded Plugin • 15 min

Simulating Agents in CHARDPT
Video • 7 min

Simulating GAME Agents in Conversation
Ungraded Plugin • 10 min

Module 4 Agent Design
Ungraded Plugin • 15 min

Agent Loop Customization
Ungraded Plugin • 15 min

Implementing GAME in Code
Ungraded Plugin • 15 min

Try Out the Agent Framework
Reading • 10 min

How Your Agent Communicates with the LLM: The Agent Language
Ungraded Plugin • 15 min

Putting It All Together: Document Your Code with a RAGGME Agent
Ungraded Plugin • 15 min

Module 4

Agent Tool Management

Keeping Agent Tools Up to Date with Python Documenter
Ungraded Plugin • 15 min

Tool Organization for Agents
Ungraded Plugin • 15 min

Enhancing Our RAGGME Agent
Ungraded Plugin • 15 min

Try Out the RAGGME Agent with the Decorator
Reading • 10 min

Module 5

Refactoring How Software is Built in the Age of AI Agents

Build the Impossible with AI Agents
Video • 2 min

Refactoring How We Teach Innovation
Video • 4 min

The Inventory Management Agent
Ungraded Plugin • 15 min

Hallucinations: A New Form of Computing
Video • 7 min

New Ways to Access and Extract Information
Video • 3 min

Implementing GAME in Code

Building a Simple Agent Framework 3

Let's go back to the file agent that we built earlier. The original implementation uses direct function calls and a lot of conditional logic in the agent loop. Let's redo the implementation using our new framework.

Define the Goals

First, let's define some goals for our file explorer agent:

```
# Define clear goals for the agent
goals = [
    Goal(
        priority1,
        name="Explore Files",
        description="Explore files in the current directory by listing and
        reading them"
    ),
    Goal(
        priority2,
        name="Terminate",
        description="Terminate the session when tasks are complete with a
        helpful summary"
    )
]
```

Create Actions Using the Framework

Next, let's convert our tool functions into properly structured Action s in our AgentRegistry:

```
def list_files() -> list[str]:
    """List files in the current directory."""
    return os.listdir(".")

def read_file(file_name: str) -> str:
    """Read a file's contents."""
    try:
        with open(file_name, "r") as file:
            return file.read()
    except FileNotFoundError:
        return f"Error: {file_name} not found."
    except Exception as e:
        return f"Error: {str(e)}"

def terminate(message: str) -> str:
    """Terminate the agent loop and provide a summary message."""
    return message

# Create and register the actions
action_registry = ActionRegistry()

action_registry.register(Action(
    name="list_files",
    function=list_files,
    description="Returns a list of files in the directory.",
    parameters={},
    terminal=False
))

action_registry.register(Action(
    name="read_file",
    function=read_file,
    description="Reads the content of a specified file in the directory.",
    parameters={
        "type": "object",
        "properties": {
            "file_name": {"type": "string"}
        },
        "required": ["file_name"]
    },
    terminal=False
))

action_registry.register(Action(
    name="terminate",
    function=terminate,
    description="Terminates the conversation. Prints the provided message for
    the user.",
    parameters={
        "type": "object",
        "properties": {
            "message": {"type": "string"},
        },
        "required": ["message"]
    },
    terminal=True
))
```

Create and Run the Agent

Now we can put it all together:

```
# Create the agent
file_explorer_agent = Agent(
    goals=goals,
    agent_language=agent_language,
    action_registry=action_registry,
    generate_response=generate_response,
    environment=environment
)

# Run the agent
user_input = input("What would you like me to do? ")
final_memory = file_explorer_agent.run(user_input, max_iterations=50)

# Print the final conversation (if desired)
for item in final_memory.get_memory():
    print(f"[{item['type'].upper():}] {item['content']}")
```

Complete Implementation

Here's the full implementation using the GAME framework:

```
def main():
    # Define the agent's goals
    goals = [
        Goal(
            priority1,
            name="Explore Files",
            description="Explore files in the current directory by listing and
            reading them"
        ),
        Goal(
            priority2,
            name="Terminate",
            description="Terminate the session when tasks are complete with a
            helpful summary"
        )
    ]

    # Define tool functions
    def list_files() -> list[str]:
        """List files in the current directory."""
        return os.listdir(".")

    def read_file(file_name: str) -> str:
        """Read a file's contents."""
        try:
            with open(file_name, "r") as file:
                return file.read()
        except FileNotFoundError:
            return f"Error: {file_name} not found."
        except Exception as e:
            return f"Error: {str(e)}"

    def terminate(message: str) -> str:
        """Terminate the agent loop and provide a summary message."""
        return message

    # Create action registry and register actions
    action_registry = ActionRegistry()

    action_registry.register(Action(
        name="list_files",
        function=list_files,
        description="Returns a list of files in the directory.",
        parameters={},
        terminal=False
    ))

    action_registry.register(Action(
        name="read_file",
        function=read_file,
        description="Reads the content of a specified file in the directory.",
        parameters={
            "type": "object",
            "properties": {
                "file_name": {"type": "string"}
            },
            "required": ["file_name"]
        },
        terminal=False
    ))

    action_registry.register(Action(
        name="terminate",
        function=terminate,
        description="Terminates the conversation. Prints the provided message
        for the user.",
        parameters={
            "type": "object",
            "properties": {
                "message": {"type": "string"},
            },
            "required": ["message"]
        },
        terminal=True
    ))

    # Define the agent language and environment
    agent_language = AgentFunctionCallingActionLanguage()
    environment = Environment()

    # Create the agent
    file_explorer_agent = Agent(
        goals=goals,
        agent_language=agent_language,
        action_registry=action_registry,
        generate_response=generate_response,
        environment=environment
    )

    # Run the agent
    user_input = input("What would you like me to do? ")
    final_memory = file_explorer_agent.run(user_input, max_iterations=50)

    # Print the termination message (if any)
    for item in final_memory.get_memory():
        print(f"[{item['type'].upper():}] {item['content']}")

if __name__ == "__main__":
    main()
```

Key Differences and Benefits

By converting our agent to the GAME framework, we gain several benefits:

- Better Organization:** Each component has a clear purpose and is separated from others.
- Reusability:** We can swap out components (like the actions or environment) without changing the core logic.
- Extensibility:** New goals and actions can be added easily.
- Standard Interface:** Using the Agent class gives us a consistent way to interact with different agents.
- Memory Management:** The framework handles memory updates automatically.

This structure also makes it easier to understand and maintain the code, especially as your agent grows in complexity.

Using the Agent

Once implemented, you can use your file explorer agent like this:

```
What would you like me to do? Tell me what Python files are in this directory
and summarize how they fit together.

Agent thinking...
Agent decision: I'll help you explore the Python files in this directory.

{"tool_name": "list_files", "args": {}}

Action Result: {"tool_executed": True, "result": ["file1.py", "file2.py",
"note.py", ...], "timestamp": "2025-03-07T12:34:56+0000"}

{"tool_name": "read_file", "args": {"file_name": "file1.py"}}

Action Result: {"tool_executed": True, "result": "# This is file1.py\n\ndef
hello_world():\n    print('Hello, World!')\n\nif __name__ == '__main__':\n    hello_world()", "timestamp": "2025-03-07T12:34:56+0000"}

[Additional file readings...]

{"tool_name": "terminate", "args": {"message": "I've explored all Python files
in this directory. Here's a summary: file1.py contains a simple hello-world
function, file2.py implements a calculator class, and main.py imports both files
and uses their functionality."}}

...
```

This structured approach makes it much easier to develop, maintain, and extend your agents over time.

Expand

Completed

Go to next item